A Constraint-Driven Relational Database Framework for Robust Digital Retail Management

Zilin Yuan*

Department of Computer Science, Rutgers University, Piscataway, NJ 08854, United States

Abstract. With the rapid development of the digital retail industry, the importance of building a stable and scalable database system has become increasingly significant, which is an inevitable requirement for the success of enterprises. Many small and medium-sized enterprises and e-commerce platforms have not yet achieved the goal of data integrity and inventory synchronization due to suboptimal data models. This paper focuses on the analysis of the design of the relational database, and introduces the advantages of the entity relationship model. The system architecture is constructed with the help of this model, and the third normal form is achieved to eliminate data redundancy and transitive dependency. In order to achieve sustainable development and reduce the occurrence of downtime, flexible reserved columns are set in the system architecture. In order to evaluate the integrity of the database, a comparative experimental framework is built, and the baseline database and the database with strong constraints are compared. It is found that the constraint-driven design has a higher anomaly interception rate, while the baseline database fails to intercept any of them. In the case of 100,000 transaction records, the system is evaluated for concurrent query performance, and the constraint-driven schema is considered to be reliable.

1 Introduction

In the process of rapid development of information technology, the retail landscape has been reshaped, driving a shift toward dynamic, data-driven online platforms [1]. In the process of building a digital retail ecosystem, the operation of the core content is more complex, from browsing the catalog and secure checkout to inventory restocking and invoice settlement. In the development of the industry, Amazon and Alibaba have used NoSQL architecture to deal with a large number of transactions, but small and medium-sized enterprises have not adopted this solution, because it will consume too many resources, and the complexity is relatively high. When small and medium-sized enterprises build their own platforms, they should not blindly follow the open source architecture, but should lay a good foundation for data, so as to avoid the occurrence of vulnerabilities [2]. For small and medium-sized enterprises, they should be able to design a database from scratch to ensure that the operation is accurate and

* Corresponding author: zilin.yuan@rutgers.edu

© The Authors, published by EDP Sciences. This is an open access article distributed under the terms of the Creative Commons Attribution License 4.0 (https://creativecommons.org/licenses/by/4.0/).

the cost is low. Stonebraker et al. pointed out that the relational model and SQL are the most reliable in the operation of transactional systems, and they are also the dominant choice, which can achieve the goal of maintainability and accuracy [3]. With the rapid development of e-commerce, the data storage structure should be changed, and the system requires scalability and availability. In the development of e-commerce, the full stack framework has been widely used, but there are still many problems with the management of data. If the database architecture is not planned, the system will be affected by data redundancy and inventory desynchronization, resulting in a series of problems. Related studies have demonstrated that if the data model is not optimized, it will be difficult to deal with a large number of items and ensure the accuracy of transactions [4]. At the same time, it should also be noted that if the system only relies on application layer code to manage business logic, it is not safe. and if the database is not strictly constrained, it will lead to serious consequences in the process of system failure and concurrent transactions. Database design is a key factor in the success of management information systems.

To address the limitations of the existing platform, this paper focuses on the design and construction of the database, introduces physical constraints into the database design. This research is different from the traditional method, introduces the extensibility mechanism, and builds a reasonable experimental framework to evaluate the integrity of the database data. This paper evaluates the robustness of the system by simulating the operation of the real world and injecting data with logical defects into the database.

In this research, three aspects are focused on. Firstly, a constraint architecture is constructed, which can map the operational risks of e-commerce to the database, including duplicate registration, orphaned transactions, overselling, and so on, and will be implemented at the database level, which can provide reference for the development of SMEs. Secondly, the extensibility strategy is introduced, and the reserved columns are allocated in advance, which can achieve schema evolution without downtime. Thirdly, the experimental framework is constructed to evaluate the effectiveness of the constraint: injecting 150 abnormal records into the baseline database and the database with constraints, analyzing the data integrity, and providing empirical evidence for the improvement of the effectiveness of the database.

The remainder of this paper is organized as follows: The second section describes the logical schema design, conceptual model construction, and data anomaly testing in detail. The third section analyzes the query performance and test results. The fourth section focuses on the significance of the research results, and the fifth section summarizes the whole text and puts forward suggestions for subsequent improvement.

2 Method

In this research, the method is used to design the relational schema, to store the data flexibly, and to test the framework to ensure the stability of the system.

2.1 Materials and Tools

MySQL is used as the relational database management system in this study. In the process of conceptual design, an ER diagram is introduced to describe the business requirements, and the relationship between the attributes of the entity is described in the diagram, including the relationship between the number of entities, and the cardinality is defined as 1: N and M: N. In the process of physical implementation, the standard SQL data definition language is used to define the data, and the data query language is used to carry out experimental verification.

2.2 Conceptual and Logical Design

The architectural design of the system is based on the transformation from conceptual model to logical model.

2.2.1 Conceptual Design (ER Diagram)

In order to grasp the core business entities, the conceptual model is constructed. As shown in Figure 1, the system includes six key entities, namely, Shoppers, Staff, Items, Purchases, Invoices, and Vendors. In order to solve the M: N relationship between Purchases and Items, a junction entity (Purchase_Line_Items) is introduced, which can record the purchase quantity and the price locked at the time of purchase. In addition, the relationship between Purchases and Invoices is 1:1, which can improve billing accuracy.

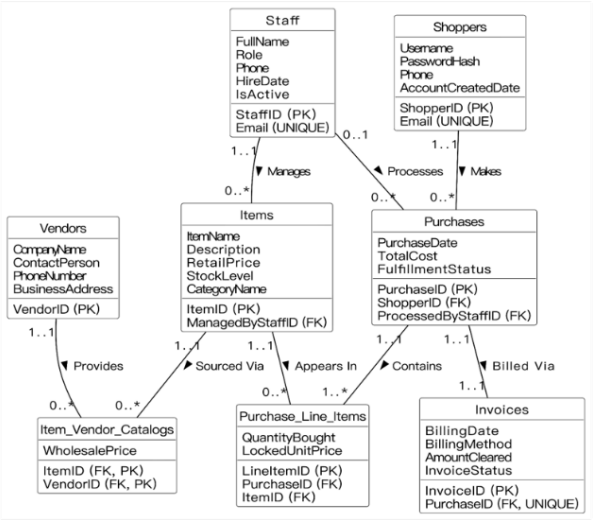


Fig. 1. ER Diagram.

2.2.2 Logical Design (Relational Schema)

The ER diagram is transformed into a logical relationship schema, which includes 8 tables, each of which sets a primary key and a foreign key to ensure the integrity of the reference.

```
Staff (StaffID (PK), FullName, Role, Phone, HireDate, IsActive, Email
(UNIQUE))
Shoppers (ShopperID (PK), Username, PasswordHash, Phone,
AccountCreatedDate, Email (UNIQUE))
Vendors (VendorID (PK), CompanyName, ContactPerson, PhoneNumber,
BusinessAddress)
Items (ItemID (PK), ItemName, Description, RetailPrice, StockLevel,
CategoryName, ManagedByStaffID (FK), Ext_JSON_Data)
Purchases (PurchaseID (PK), PurchaseDate, TotalCost,
FulfillmentStatus, ShopperID (FK), ProcessedByStaffID (FK))
Invoices (InvoiceID (PK), BillingDate, BillingMethod, AmountCleared,
InvoiceStatus, PurchaseID (FK, UNIQUE))
Item_Vendor_Catalogs (ItemID (FK, PK), VendorID (FK, PK),
WholesalePrice)
Purchase_Line_Items (LineItemID (PK), QuantityBought, LockedUnitPrice,
PurchaseID (FK), ItemID (FK))
```

2.2.3 Business Rules and Constraints

In order to prevent operational errors, physical constraints are set for several business rules:

Inventory integrity: This constraint is set for Items. StockLevel, which means that the inventory can not be less than zero.

Unique identity: The Email field in the Shoppers table is constrained by a unique index to prevent duplicate registration.

Soft-Delete Policy: Referential integrity actions (ON DELETE RESTRICT) prevent the physical removal of shopper records if they are associated with existing purchases.

In order to make the above business rules more intuitive, the SQL DDL code is used to create the table, and the database is used to set the constraint conditions:

```
CREATE TABLE Items (
  ItemID INT AUTO_INCREMENT PRIMARY KEY,
  ItemName VARCHAR(100) NOT NULL,
  RetailPrice DECIMAL (10, 2) NOT NULL,
  StockLevel INT NOT NULL,
  CategoryName VARCHAR(50),
  ManagedByStaffID INT,
  Ext_JSON_Data JSON DEFAULT NULL,
  CONSTRAINT chk_stock_level CHECK (StockLevel >= 0),
  FOREIGN KEY (ManagedByStaffID) REFERENCES Staff (StaffID)
);
```

2.3 Database Normalization and Flexible Design

In order to achieve the goal of system longevity and data integrity, the relational structure should be standardized to 3NF [5]. The structural refinement is conducive to the development of e-commerce enterprises, which can ensure the consistency and availability of data without degrading system performance [4]. In the process of system design, transitive dependency and partial dependency are eliminated, and transaction records are separated from the shopper profile, so as to reduce the risk of update anomalies. This is an inevitable requirement for the processing of massive data in the retail environment [6].

In order to verify whether 3NF is satisfied, the functional dependency relationship should be analyzed. For example, in the Items table, all non-prime attributes are fully dependent on the primary key (ItemID → {ItemName, RetailPrice, StockLevel, CategoryName}). At the same time, transitive dependency should also be eliminated. CategoryName serves as a descriptive attribute and does not act as a determinant for other non-prime attributes. Through such a relational mapping, data integrity can be guaranteed and update anomalies can be avoided.

In order to ensure the integrity of the data, it is necessary to verify whether all eight tables meet the requirements of 3NF. In the Staff table, all non-prime attributes can be directly dependent on StaffID, and there is no transitive dependency. In the Shoppers table, all attributes can be described by ShopperID. In the Vendors table, all attributes can be described by VendorID. In the Purchases table, there are no transitive or partial dependencies. The fulfillment status and total cost can describe the transaction itself, and ProcessedByStaffID and ShopperID are foreign keys. In the Invoices table, the relationship between the invoice and the purchase is 1:1, and the invoice status, billing method, billing date, and other attributes are determined by InvoiceID. In the Item_Vendor_Catalogs table, there is only one non-prime attribute (WholesalePrice), which is fully dependent on the composite primary key (ItemID, VendorID), and the requirements of 2NF and 3NF are met. In the Purchase_Line_Items table, QuantityBought and LockedUnitPrice are determined by LineItemID, with PurchaseID and ItemID serving as foreign keys.

There is a need to make a reasonable decision in the design, that is, to regard CategoryName as an attribute of the Items table, and not to normalize it into an independent

entity. In the strict sense, if CategoryName can be used to determine other attributes, it will form a transitive dependency, which does not conform to 3NF. In the current schema, CategoryName is only a descriptive label and does not depend on other attributes, so there is no transitive dependency. This is a trade-off between engineering and design: in the case of small and medium-sized enterprises, if there is no change in category metadata, the additional join will lead to a decline in efficiency, and the normalization benefits are not significant. If there are more and more attributes in the category in the development of business, it is necessary to set up a special category table, and the foreign key reference of Items should be considered.

In the process of standardization, the strategy of flexible design should also be taken into account, so that the system can have a longer service life. In order to make the digital retail more dynamic, in the process of building the architecture, a "Reserved Column" should be set in the table with high traffic, and the data dictionary space should be prepared for the next stage of business logic expansion, which will not lead to the change of the structure of the production database. The data integrity can be guaranteed by combining the schema with physical constraints, which outperformed the application layer logic in protecting data integrity [7].

The column reservation strategy offers significant extensibility advantages, but it is necessary to make a reasonable choice according to the actual situation. First, if some columns are allocated in advance, but they have been empty for a long time, it will cause marginal storage overhead, which is more prominent in the purchase table. Second, the JSON type extension field (Ext_JSON_Data) can not be indexed directly. If you want to filter dynamic attributes, you need to use the generated column of MySQL to generate virtual columns. Third, the clarity of column semantics depends on the governance of the schema, and if there is no data dictionary, the column will be ambiguous, and the business meaning will conflict after multiple iterations. Despite the above shortcomings, this method is still widely used in the SME retail system, because the risk of stopping production is greater than the cost of storage.

2.4 Experimental Setup and Anomaly Testing

In order to ensure the integrity of the data and the effectiveness of the constraints, the simulated test environment is constructed. The mock dataset contains 100,000 purchase transactions, approximately 300,000 line item records, 5,000 items, and 1,000 registered shoppers, which can effectively test the defense mechanism of the database. The simulated test environment is built on a computer with 32 GB of memory and a Windows 11 system, equipped with an Intel Core Ultra9 processor, and MySQL 8.0 is used as the database engine. Two databases are set up: one is the baseline version, which only retains the table structure and deletes all constraints, and the other is the constrained version, which has a complete schema design. In order to enhance the statistical significance, two instances are tested with 50 different test cases for each category of anomalies, including:

1. Data Redundancy: Attempts to insert duplicate shopper registration emails.
2. Unreasonable Numerical Values: Invalid operations, such as updating inventory to negative values.
3. Incomplete data: In the process of evaluation, some critical foreign keys are missing, such as assigning the purchase to a shopper who does not exist, and the integrity of the reference can not be guaranteed.

Finally, to assess system performance, a complex multi-table query is designed to integrate a large amount of data from the Purchases, Purchase_Line_Items, Items, and Invoices tables.

3 Results

In this chapter, the robustness of the database is analyzed and evaluated. In the experiment, a simulated data set is constructed to analyze whether the underlying constraints can intercept abnormal data.

3.1 Database Constraint Validation

In order to accurately grasp the data integrity risk caused by the lack of database constraints, a comparative injection experiment was carried out. In this experiment, the baseline database was not restricted, but the table structure was the same as that of the database, and there were no foreign keys, unique indexes, and check constraints in the database. In the experiment, the test cases were set to 150, and the test cases were divided into 3 categories, 50 each category. The test results are shown in Table 1, which shows that the baseline database can not resist the intrusion of 150 abnormal records, and the data integrity rate is only 0%, which means that the user table is polluted, the inventory state is corrupted, and the transaction records are orphaned. In the process of application, the system of SMEs will encounter such problems if there are no database constraints. The constrained version can intercept 150 abnormal records, which means that the constraint architecture is reasonable, and the data integrity risk can be eliminated at the database level.

Table 1. Data Corruption Risk: Unconstrained vs. Constrained Database Under Anomalous Injection.

Test Category	Tested Constraint	Unconstrained DB (Written)	Constrained DB (Blocked)	Interception Rate
Data Redundancy	UNIQUE Index	50/50 (0% blocked)	50/50 (100%)	100%
Numerical Validity	CHECK Constraint	50/50 (0% blocked)	50/50 (100%)	100%
Missing Data	Foreign Key (FK)	50/50 (0% blocked)	50/50 (100%)	100%
Total	—	**150/150 (0%)**	**150/150 (100%)**	**100%**

Unique constraint test: When a duplicate email address is inserted, the system will report an error and intercept the operation.

Numerical validity test: The negative value "-5" is injected into the Items. StockLevel field. The database system can immediately report an error and will not allow the creation of a negative inventory.

Reference integrity test: An attempt was made to delete a shopper account associated with historical purchase records. The system rejected the deletion, which proves that the financial record protection mechanism is reliable.

3.2 Query Performance Analysis

In this study, a complex multi-table query is constructed, which involves four tables: Purchases, Purchase_Line_Items, Items, and Invoices, and can generate reports on stock and revenue. The data set is scaled to 5,000 items, 300,000 line items, and 100,000 transactions. The stress testing script can be used to query both the constrained and unconstrained databases at the same time, and the number of threads is set to 100, 300, and 500. As can be seen from Table 2, the constrained version outperformed the unconstrained version across all concurrency levels. At 100 concurrent threads, the constrained database averaged 993.9 ms and the unconstrained database averaged 1,091.3 ms, with a 9.8% overhead on the unconstrained side. Under heavier load, the performance gap widened: when the number of threads reached 500, the constrained database averaged 2,637.3 ms, while the unconstrained

database averaged 3,941.2 ms, which means the unconstrained version was 49.4% slower. There were no failed requests in the two versions under the same concurrency level. The reason for this counterintuitive result is that the Invoices table has a UNIQUE index (uq_invoices_purchase), which can be used for eq_ref lookups by the query optimizer in the constrained version, but the index does not exist in the unconstrained version, so the query optimizer can only fall back to full table scans. These findings show that the implementation of database constraints will not lead to performance penalties, but can instead bring benefits to query performance, since the implementation of index-assisted execution paths will be more efficient. EXPLAIN analysis also shows that the Purchase_Line_Items table can use the fk_pli_purchase index to access the data, while the Purchases table can only scan the entire table, which means that adding an index on PurchaseDate would improve performance in the process of production deployment.

Table 2. Query performance comparison of constrained and unconstrained databases (100,000 transaction records, single-node MySQL 8.0).

Conc.	Constrained Avg	Constrained P99	Unconstrained Avg	Unconstrained P99	Failed (C/U)	Overhead %
100	993.9 ms	1,728.6 ms	1,091.3 ms	1,852.1 ms	0 / 0	+9.8%
300	1,865.4 ms	4,891.7 ms	2,350.0 ms	5,301.1 ms	0 / 0	+26.0%
500	2,637.3 ms	8,287.2 ms	3,941.2 ms	10,867.2 ms	0 / 0	+49.4%

4 Discussion

4.1 Analysis of Findings

The experimental results indicate that database-level constraint enforcement plays a key role in maintaining data consistency. The logical conversion can be carried out in the design phase to achieve 3NF, and the constraint is set according to the actual situation of retail enterprises, which is different from the traditional solution that only maintains logic through application code. Using constraint mechanism in the database physical layer can bring higher resilience, and will not be affected by network and application status. When the stock level is checked only through the application code (such as Node.js validation middleware), network latency and high-concurrency checkouts can cause race conditions, which will lead to phantom reading and overselling [8]. In contrast, the MySQL engine performs row-level locking to guarantee atomicity, and invalid transactions are rejected at the database level, so the inventory remains consistent regardless of the application status or the concurrent traffic. It should be noted that the comparison between the constrained database and the unconstrained database shows that the constrained version does not incur a significant performance penalty. Instead, the constraint design introduces a unique index, which can help query execution and promote query performance. At 500 concurrent threads, the constrained version is 49.4% faster than the unconstrained baseline, which challenges the common assumption that constraints trade performance for integrity, and confirms that constraint-driven schema design should be adopted in the retail system of SMEs.

4.2 System Recommendations and Flexible Design

From the test results, it is recommended that SMEs use the RDBMS architecture to ensure ACID properties. The most prominent feature of this project is the reserved column, which has good compatibility and can quickly support new business without changing the physical schema, enabling zero-downtime upgrades.

4.3 Limitations and Future Improvements

Although the system is highly robust in the test, there are some shortcomings in many aspects. At present, the test framework is based on a MySQL 8.0 environment with 100,000 transaction records. In this case, the database is constrained, and the query of the four tables is more complex, with response times between 993.9 ms and 2,637.3 ms across 100 to 500 concurrent threads, which reflects the characteristics of analytical workloads on a single node deployment. In this study, the performance of the system is analyzed on the basis of the scale of the system and the configuration of the system, and these performance benchmarks may not generalize to high-concurrency production environments without further validation. In the EXPLAIN analysis, the full table scanning operation is carried out on the Purchases table, and the filesort operation is also carried out, which requires the system to optimize the index before the system is deployed on a large scale. In order to solve the problem of distributed consistency in massive concurrency, the database should be deployed in a cluster environment in the next stage of verification, and the stress test should be carried out with more than 10,000 concurrent users. In the next stage of database development, the reserved column mechanism should be leveraged to build flexible JSON dynamic fields, which can balance structured storage and unstructured agility. In the next stage of system iteration, an in-memory caching layer should be integrated into the system, such as Redis, to absorb high-concurrency read requests, and reduce the load of MySQL RDBMS in the peak period [9, 10].

5 Conclusion

This paper focuses on the study of high-availability relational databases, and introduces physical constraints and third normal form into the system. In this way, it can not only solve the problem of inventory desynchronization, but also avoid duplicate registration and orphaned transactions. In the experiment, the database with constraints and the database without constraints are compared, and the former has a 100% anomaly interception rate, which demonstrates that constraint enforcement outperforms the unconstrained baseline. The test data set has 100,000 records, and the number of threads is between 100 and 500. It is found that the query response characteristics of the database can meet the requirements of users in the process of operation, and the request failure rate is 0. In this study, a column reservation mechanism is set to make the system have strong scalability and evolve without interruption. Generally speaking, the research results of this paper can provide a reference for the construction of constraint-driven and scalable infrastructure for small and medium-sized enterprises.

References

1. E. Dritsas and M. Trigka, Database Systems in the Big Data Era: Architectures, Performance, and Open Challenges, IEEE Access, 13, 95068–95084 (2025). https://doi.org/10.1109/ACCESS.2025.3572059
2. H. Shwetha, D. Prajwal, and S. Sridharan, From MongoDB to React: Unleashing the Power of MERN in E-commerce, in Proc. 2024 Int. Conf. Emerg. Technol. Comput. Sci. Interdiscip. Appl. (ICETCS), pp. 1–6 (2024). https://doi.org/10.1109/ICETCS61022.2024.10543521
3. M. Stonebraker and A. Pavlo, What Goes Around Comes Around... And Around..., SIGMOD Rec., vol. 53, no. 2, pp. 21–37 (2024). https://doi.org/10.1145/3685980.3685984
4. D. Patel and M. Shah, Data Modeling Practices for E-Commerce, Grenze Int. J. Eng. Technol., vol. 10, no. 1, pp. 614–622 (2024).

5. A. H. Chillón, M. Klettke, D. S. Ruiz, and J. G. Molina, A Generic Schema Evolution Approach for NoSQL and Relational Databases, IEEE Trans. Knowl. Data Eng., vol. 36, no. 7, pp. 2774–2789 (2024). https://doi.org/10.1109/TKDE.2024.3362273
6. H. Dong, C. Zhang, G. Li, and H. Zhang, Cloud-Native Databases: A Survey, IEEE Trans. Knowl. Data Eng., vol. 36, no. 12, pp. 7772–7791 (2024). https://doi.org/10.1109/TKDE.2024.3397508
7. H. Huang, B. Shen, L. Zhong, and Y. Zhou, Protecting Data Integrity of Web Applications with Database Constraints Inferred from Application Code, in Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS), vol. 2, pp. 632–645 (2023). https://doi.org/10.1145/3575693.3575699
8. A. Chen, Y. Kwon, and K. H. Lee, Racedb: Detecting Request Race Vulnerabilities in Database-Backed Web Applications, in Proc. 2025 IEEE Symp. Secur. Privacy (SP), pp. 939–955 (2025). https://doi.org/10.1109/SP61157.2025.00029
9. Y. Zhu, T. Xia, T. Zhu, Z. Zhao, K. Li, and X. Hu, RAPO: An Automated Performance Optimization Tool for Redis Clusters in Distributed Storage Metadata Management, IEEE Access, vol. 13, pp. 58060–58074 (2025). https://doi.org/10.1109/ACCESS.2025.3556240
10. V. Repin and A. Sidorov, Distributed Caching System with Strong Consistency Model, Front. Comput. Sci., vol. 7, 1511161 (2025). https://doi.org/10.3389/fcomp.2025.1511161